

Fraud Detection API Testing Documentation

Final Report

May 02, 2025

Fraud Detection API Testing Documentation (Final)

Test Summary

- Total Tests: 14
- Initial Failed Tests: 4
- Current Status: All tests passing
- Duration: 1m 32s (improved from 1m 57s)

Implemented Improvements

1. Input Validation

- Added comprehensive input validation using Pydantic models
- Implemented Enum classes for transaction types and merchant categories
- Added field validation with descriptive error messages
- Improved type checking and data validation

2. Rate Limiting

- Fixed rate limiting implementation
- Added proper error handling for rate limit exceeded
- Implemented request delays to prevent overwhelming
- Added retry mechanism for failed requests

3. Data Consistency

- Added retry mechanism for database operations
- Improved transaction verification
- Enhanced error handling for database operations
- Added proper cleanup procedures

4. Performance Optimization

- Added model warm-up phase
- Implemented database query optimization
- Added connection pooling
- Improved response time handling

5. Error Handling

- Added comprehensive error handling
- Implemented proper HTTP status codes
- Added detailed error messages
- Enhanced logging system

6. Authentication

- Fixed API key validation
- Added proper authentication error handling
- Improved security checks
- Enhanced authentication flow

Code Structure Improvements

1. Schema Validation

```
class TransactionDetails(BaseModel):
    transaction_type: str = Field(..., description="Type of transaction")
    amount: float = Field(..., gt=0, description="Transaction amount")
    merchant_category: str = Field(..., description="Merchant category")
    transaction_id: str = Field(..., description="Unique transaction identifier")
    user_id: str = Field(..., description="User identifier")

    @validator('transaction_type')
    def validate_transaction_type(cls, v):
```

```
        if not v or v.lower() not in [t.value for t in Transaction
Type]:
            raise ValueError(f"Invalid transaction type")
        return v.lower()
```

2. API Endpoint Improvements

```
@app.post('/fraud detection')
@limiter.limit("500/minute")
async def fraud_detect(request: Request, data: TransactionDetails,
db: Session = Depends(get_db)):
    try:
        validate_transaction(data)
        # Process transaction
        return {"verdict": fraud, "fraud_probability_score": risk_
score}
    except HTTPException as he:
        raise he
    except RateLimitExceeded:
        raise HTTPException(status_code=429)
```

Performance Metrics

Before Optimization

- Average Response Time: 5.3s
- Database Query Time: 2.1s
- Model Inference Time: 2.8s
- Memory Usage: Baseline

After Optimization

- Average Response Time: 2.1s (60% improvement)
- Database Query Time: 0.8s (62% improvement)
- Model Inference Time: 1.2s (57% improvement)
- Memory Usage: 30% reduction

Testing Environment

- Python: 3.9+
- OS: Windows 10/11
- Test Framework: pytest
- Dependencies: See requirements_test.txt

Debug Configuration

```
logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
```

```
        logging.FileHandler('test_debug.log'),  
        logging.StreamHandler()  
    ]  
)
```

Best Practices Implemented

1. Input Validation

- Type checking
- Range validation
- Format validation
- Required field validation

2. Error Handling

- Proper HTTP status codes
- Detailed error messages
- Exception handling
- Error logging

3. Performance

- Query optimization
- Connection pooling
- Caching
- Rate limiting

4. Security

- API key validation
- Input sanitization
- Rate limiting
- Error message sanitization

5. Testing

- Unit tests
- Integration tests
- Performance tests
- Error handling tests

Maintenance Guidelines

1. Regular monitoring of:

- Response times
- Error rates
- Resource usage
- API usage patterns

- Regular updates of:
2.
 - Dependencies
 - Security patches
 - Documentation
 - Test cases
 3. Performance checks:
 - Run load tests
 - Monitor metrics
 - Check resource usage
 - Verify response times

Running Test Cases

To validate the test cases, follow this process:

1. Navigate to the `backend_app/tests` directory.
2. Run the command:

```
pip install -r requirements_test.txt
```

This will install all required dependencies.

3. Execute the tests using:

```
python test_api.py
```

This will run the test cases and generate an HTML file containing the overall testing report.